

Game-Dev Agent Skills

A field guide to the 75 skills installed in `.claude/skills/` from awesome-gamedev-agent-skills: what each one does and when to reach for it.

How to use them	
Let the router pick	Describe what you're building ("add a double jump to my Godot player"). The router detects the engine from files like <code>project.godot</code> or <code>package.json</code> and loads the matching skills.
Call one directly	Type <code>/skill-name</code> , e.g. <code>/godot-gdscript</code> or <code>/game-feel</code> , to load a specific skill.
Combine them	Engine skills cover the API; discipline and genre skills cover the design. E.g. <code>platformer + godot-2d-movement + game-feel</code> .

Contents

Start here: the router	1
Godot	16
Unity	8
Unreal Engine	6
Web engines	6
Other engines	10
Disciplines (engine-neutral)	15
Genres	9
Workflows	4

Source: github.com/gamedev-skills/awesome-gamedev-agent-skills · Apache-2.0. Descriptions are taken from each skill's SKILL.md.

Start here: the router

Loads the right skills for you based on your engine and task. · 1 skill

router

WHAT IT DOES

Routes any game-development request to the right specialized skill(s): it detects the engine (Godot, Unity, Unreal, Bevy, Phaser, Pixijs, three.js, LÖVE, pygame, Roblox) and the task, then reads the chosen skill before acting.

WHEN TO USE

Use to make a game or to decide which skill applies — for players, levels, enemies, shaders, art direction, sprites, tiles, textures, 3D assets, UI/UX, cameras, game feel, physics, input, audio, saving, multiplayer, AI, dialogue, procedural generation, or performance, for genres (platformer, roguelike, RPG, FPS, tower-defense, card game, visual novel, survival-crafting, puzzle), and for shipping (game jam, Steam, itch). Start here when unsure which gamedev skill to use.

Invoke: **/router** · or just describe your task

Godot

Godot 4.7 skills: GDScript/C#, scenes, physics, UI, export and more. · 16 skills

godot-2d-movement

WHAT IT DOES

Implement 2D kinematic character movement in Godot 4.7 with CharacterBody2D and move_and_slide(): platformer run/jump with gravity, top-down 8-direction motion, slope handling, and reading collisions.

WHEN TO USE

Use when coding a 2D player or enemy controller, a platformer or top-down character, or fixing move_and_slide()/ is_on_floor() behavior in a .tscn with a CharacterBody2D.

Invoke: **/godot-2d-movement** · or just describe your task

godot-3d-essentials

WHAT IT DOES

Set up a Godot 4.7 3D scene: Node3D transforms, Camera3D, lighting (DirectionalLight3D/OmniLight3D), WorldEnvironment for sky/ambient/tonemap/post, MeshInstance3D materials, and GridMap for tile-based 3D levels.

WHEN TO USE

Use when building a 3D scene in a Godot project, placing cameras/lights, configuring environment and post-processing, or working with Node3D/.tscn 3D content and GridMap.

Invoke: **/godot-3d-essentials** · or just describe your task

godot-animation

WHAT IT DOES

Animate in Godot 4.7 three ways: AnimationPlayer for keyframed clips (incl. call and signal tracks), AnimationTree with state machines and blend spaces for character animation, and Tween for short procedural/UI tweens via create_tween().

WHEN TO USE

Use when working with AnimationPlayer/AnimationTree nodes in a .tscn, blending character states, sprite-sheet animation, or code-driven Tweens.

Invoke: **/godot-animation** · or just describe your task

godot-audio

WHAT IT DOES

Play and mix audio in Godot 4.7: AudioStreamPlayer (2D/3D variants), audio buses with volume/mute and effects, music vs SFX routing, db/linear volume, and precise sync-to-beat playback timing.

WHEN TO USE

Use when playing sounds or music in a Godot project, routing AudioStreamPlayer nodes to buses, adjusting bus volume via AudioServer, or syncing gameplay to the beat.

Invoke: **/godot-audio** · or just describe your task

godot-csharp

WHAT IT DOES

Use C#/.NET in Godot 4.7: partial classes extending nodes, the PascalCase lifecycle (`_Ready/_Process/_PhysicsProcess`), `[Export]` fields, `[Signal]` delegates as C# events, type-safe node lookup, and calling between C# and GDScript.

WHEN TO USE

Use when writing Godot game code in C# (.cs files, .csproj), needing the Godot .NET build, converting GDScript patterns to C#, or wiring Godot signals as C# events.

Invoke: **/godot-csharp** · or just describe your task

godot-export

WHAT IT DOES

Export and build a Godot 4.7 project for distribution: install export templates, define export presets (Windows/macOS/Linux/Web/Android), run headless command-line exports for CI, and handle web (HTML5) COOP/COEP and dedicated-server/headless builds.

WHEN TO USE

Use when exporting a Godot game, configuring `export_presets.cfg`, building for web/desktop/mobile, or automating builds from the command line.

Invoke: **/godot-export** · or just describe your task

godot-gdscript

WHAT IT DOES

Write idiomatic GDScript for Godot 4.7: static typing, the node lifecycle (`_ready/_process/_physics_process`), `@export/@onready/@tool` annotations, signals, and await for asynchronous flow.

WHEN TO USE

Use when editing .gd scripts in a Godot project (project.godot), writing or debugging GDScript, or porting 3.x GDScript to 4.x (function signatures, yield to await, export to `@export`).

Invoke: **/godot-gdscript** · or just describe your task

godot-gdscript-headless-testing

WHAT IT DOES

Run GDScript test suites from the command line with ``godot --headless``, using a SceneTree/MainLoop runner script that exits non-zero on failure so CI can gate merges.

WHEN TO USE

Use when a Godot project needs unit tests without opening the editor, when wiring a CI job (GitHub Actions or similar) that must fail the build on a failing ``gd`` test, or when a ``godot --headless`` invocation hangs, opens a window, or exits 0 despite failed assertions.

Invoke: `/godot-gdscript-headless-testing` · or just describe your task

godot-multiplayer

WHAT IT DOES

Build networked games with Godot 4.7 high-level multiplayer: set up an ENetMultiplayerPeer server/client, define RPCs with the `@rpc` annotation (call via `rpc()/rpc_id()`), set per-node multiplayer authority, and replicate state with MultiplayerSpawner and MultiplayerSynchronizer.

WHEN TO USE

Use when adding multiplayer/networking to a Godot project, writing `@rpc` functions, or syncing player/world state across peers.

Invoke: `/godot-multiplayer` · or just describe your task

godot-nodes-scenes

WHAT IT DOES

Structure a Godot 4.7 project with the scene tree and node composition: build reusable scenes, instance PackedScenes at runtime, navigate the tree safely, and register autoload singletons.

WHEN TO USE

Use when designing `.tscn` scenes, deciding how to split nodes, spawning instances with `instantiate()`, wiring autoloads, or fixing "node not found"/freed-node errors in a Godot project.

Invoke: `/godot-nodes-scenes` · or just describe your task

godot-physics

WHAT IT DOES

Use Godot 4.7 physics bodies and detection in 2D and 3D: RigidBody, StaticBody, Area, and CharacterBody; collision layers vs masks; contact/overlap signals; and raycasts (RayCast nodes and direct space-state queries).

WHEN TO USE

Use when configuring collision layers/masks, detecting overlaps with Area2D/Area3D, applying forces to a RigidBody, or casting rays in a Godot project (`.tscn` with physics bodies), including 3D projects running Jolt Physics, the default 3D engine for projects created in 4.6+.

Invoke: `/godot-physics` · or just describe your task

godot-resources

WHAT IT DOES

Design data-driven Godot 4.7 games with custom Resource classes: define typed data with `class_name + @export`, save/load `.tres/.res` files, instance and duplicate resources, and load on demand with `ResourceLoader` (incl. threaded loading).

WHEN TO USE

Use when modeling items/stats/configs as data in a Godot project, creating `.tres` resources, or working with custom Resource subclasses and `ResourceLoader/ResourceSaver`.

Invoke: `/godot-resources` · or just describe your task

godot-shaders

WHAT IT DOES

Write Godot 4.7 shaders in the Godot Shading Language: `canvas_item` shaders for 2D and spatial shaders for 3D, with vertex/fragment functions, uniforms (`source_color`, `hint_range`), `TIME/UV` animation, and screen-reading via `hint_screen_texture`.

WHEN TO USE

Use when authoring `.gdshader` files, writing fragment/vertex code, making 2D/3D visual effects, or porting 3.x shaders (`SCREEN_TEXTURE`, `hint_color`) to 4.x.

Invoke: `/godot-shaders` · or just describe your task

godot-signals-groups

WHAT IT DOES

Build event-driven, decoupled Godot 4.7 gameplay with signals and node groups: declare and emit custom signals, connect with Callables (incl. `bind/one-shot`), and broadcast to many nodes via groups and `call_group`.

WHEN TO USE

Use when wiring node communication in a Godot project, replacing tight references with signals, emitting/connecting events, or porting 3.x `connect("sig", self, "method")` code.

Invoke: `/godot-signals-groups` · or just describe your task

godot-tilemap

WHAT IT DOES

Build and edit tile-based 2D levels in Godot 4.7 with `TileMapLayer` and `TileSet`: paint layers, set up collision/navigation/custom-data on tiles, autotile with terrain sets, and read/write cells from code (`set_cell`, `get_cell_tile_data`, `local_to_map`).

WHEN TO USE

Use when working with `TileMapLayer` nodes, `.tres` `TileSets`, autotiling, or migrating a deprecated `TileMap` node to `TileMapLayer`.

Invoke: `/godot-tilemap` · or just describe your task

godot-ui-control

WHAT IT DOES

Build Godot 4.7 user interfaces with Control nodes: anchors and offsets for responsive layout, Container nodes (VBox/HBox/Grid/Margin) for automatic arrangement, Theme resources for consistent styling, and keyboard/gamepad focus navigation.

WHEN TO USE

Use when laying out a HUD, menu, or UI in a Godot project, working with Control/Container nodes, anchors, themes, or focus in a .tscn.

Invoke: `/godot-ui-control` · or just describe your task

Unity

Unity 6.3 LTS skills: C# scripting, physics, input, animation, builds. · 8 skills

unity-animation

WHAT IT DOES

Drive Unity 6.3 LTS character animation with Animator Controllers: states, transitions, parameters, blend trees, animation layers, and humanoid Avatar IK.

WHEN TO USE

Use when wiring an Animator, setting parameters from script (SetFloat/SetBool/SetTrigger), building blend trees, or when the user mentions Animator, Mecanim, state machine, blend tree, or .controller.

Invoke: **/unity-animation** · or just describe your task

unity-build-pipeline

WHAT IT DOES

Build and ship Unity 6.3 LTS players: build settings and scenes, player/quality settings, the IL2CPP vs Mono scripting backend, managed code stripping, scripted BuildPipeline.BuildPlayer, and CI/headless builds.

WHEN TO USE

Use when configuring or automating a build, choosing a scripting backend, shrinking build size, or when the user mentions Unity build, player settings, IL2CPP, code stripping, or Addressables.

Invoke: **/unity-build-pipeline** · or just describe your task

unity-csharp-scripting

WHAT IT DOES

Write Unity 6.3 LTS C# gameplay scripts: the MonoBehaviour lifecycle (Awake/OnEnable/Start/Update/FixedUpdate/LateUpdate), GameObject and component access, coroutines, and Inspector serialization.

WHEN TO USE

Use when creating or editing .cs scripts in a Unity project, or when the user mentions MonoBehaviour, Start/Update, GetComponent, SerializeField, coroutines, or "Unity script".

Invoke: **/unity-csharp-scripting** · or just describe your task

unity-input-system

WHAT IT DOES

Wire player input in Unity 6.3 LTS with the Input System package: Input Actions, action maps, the PlayerInput component, and reading values via callbacks or polling.

WHEN TO USE

Use when the project has a .inputactions asset or com.unity.inputsystem, or when the user mentions the Unity Input System, InputAction, action maps, PlayerInput, control schemes, or rebinding.

Invoke: `/unity-input-system` · or just describe your task

unity-navmesh

WHAT IT DOES

Add AI navigation in Unity 6.3 LTS: bake a NavMesh with the AI Navigation package (NavMeshSurface), move agents with NavMeshAgent.SetDestination, and handle dynamic obstacles.

WHEN TO USE

Use when setting up pathfinding, making an enemy chase the player, baking navigation, or when the user mentions NavMesh, NavMeshAgent, NavMeshSurface, NavMeshObstacle, or Unity pathfinding.

Invoke: `/unity-navmesh` · or just describe your task

unity-physics

WHAT IT DOES

Set up 3D physics in Unity 6.3 LTS: Rigidbody movement and forces, colliders, triggers vs collisions, layer-based collision, raycasts, and joints.

WHEN TO USE

Use when adding a Rigidbody, handling OnCollisionEnter/OnTriggerEnter, tuning collision layers, casting rays, or when the user mentions Unity physics, AddForce, isKinematic, or linearVelocity.

Invoke: `/unity-physics` · or just describe your task

unity-scriptableobjects

WHAT IT DOES

Architect Unity 6.3 LTS data and decoupling with ScriptableObjects: config/data assets, shared runtime variables, event channels, and runtime sets/registries.

WHEN TO USE

Use when designing data-driven systems, replacing singletons/managers, creating .asset data with CreateAssetMenu, or when the user mentions ScriptableObject, SO architecture, or data assets.

Invoke: `/unity-scriptableobjects` · or just describe your task

unity-tilemap-2d

WHAT IT DOES

Build and script 2D tilemaps in Unity 6.3 LTS: the Grid + Tilemap components, the Tile Palette, tilemap colliders, rule tiles, and runtime SetTile/GetTile painting.

WHEN TO USE

Use when painting tile levels, adding a TilemapCollider2D, using rule or animated tiles, generating tilemaps from code, or when the user mentions Unity tilemap, tile palette, rule tile, or Grid.

Invoke: **/unity-tilemap-2d** · or just describe your task

Unreal Engine

Unreal Engine 5 skills: C++, Blueprints, input, AI, VFX, packaging. · 6 skills

unreal-behavior-trees

WHAT IT DOES

Build NPC AI in Unreal Engine 5 with Behavior Trees and Blackboards: composites (Selector/Sequence), tasks, decorators, services, and running the tree from an AIController.

WHEN TO USE

Use when creating enemy/NPC AI, BT_/BB_ assets, custom BTTask or BTService nodes, or when the user mentions Behavior Tree, Blackboard, AIController, BTTask, decorator, or service.

Invoke: `/unreal-behavior-trees` · or just describe your task

unreal-blueprints

WHAT IT DOES

Build Unreal Engine 5 gameplay with Blueprint visual scripting: Blueprint Classes, the Event Graph and Construction Script, variables/functions/macros, and Blueprint communication (Cast, Interfaces, Event Dispatchers).

WHEN TO USE

Use when working in Blueprints, wiring an event graph, deciding how Blueprints talk to each other, or when the user mentions Blueprint, BP, event graph, construction script, or a Blueprint .uasset.

Invoke: `/unreal-blueprints` · or just describe your task

unreal-cpp-gameplay

WHAT IT DOES

Write Unreal Engine 5 C++ gameplay code: the UCLASS/UPROPERTY/UFUNCTION reflection macros, the Gameplay Framework (GameMode, Pawn, Character, PlayerController, Actor components), and the module Build.cs.

WHEN TO USE

Use when writing or debugging UE C++, deriving from AActor/ACharacter/ AGameModeBase, exposing properties to the editor or Blueprints, or when the user mentions Unreal C++, UCLASS, GENERATED_BODY, GameMode, ACharacter, or .Build.cs.

Invoke: `/unreal-cpp-gameplay` · or just describe your task

unreal-enhanced-input

WHAT IT DOES

Set up player input in Unreal Engine 5 with Enhanced Input: Input Actions, Input Mapping Contexts, modifiers and triggers, adding the mapping context, and binding actions by ETriggerEvent.

WHEN TO USE

Use when wiring movement/look/jump input, creating IA_/IMC_ assets, binding in C++ or Blueprints, or when the user mentions Enhanced Input, Input Mapping Context, Input Action, IA_/IMC_, or ETriggerEvent.

Invoke: `/unreal-enhanced-input` · or just describe your task

unreal-niagara

WHAT IT DOES

Create and control VFX in Unreal Engine 5 with Niagara: systems and emitters, modules and the spawn/update stages, exposed User parameters, and spawning or driving effects from Blueprints or C++.

WHEN TO USE

Use when building particle effects, NS_/NE_ assets, spawning a Niagara system at runtime, setting User parameters, or when the user mentions Niagara, VFX, or a particle system in Unreal.

Invoke: `/unreal-niagara` · or just describe your task

unreal-packaging

WHAT IT DOES

Package and ship an Unreal Engine 5 project: the Platforms menu Package Project flow, build configurations (Development vs Shipping), cooking content, packaging settings and the Game Default Map, and command-line builds with RunUAT BuildCookRun.

WHEN TO USE

Use when packaging a build, making a shipping build, cooking content, configuring packaging settings, or when the user mentions package Unreal, cook content, shipping build, or BuildCookRun.

Invoke: `/unreal-packaging` · or just describe your task

Web engines

Phaser 4, Pixijs v8 and three.js · 6 skills

phaser-arcade-physics

WHAT IT DOES

Use Phaser 4 Arcade Physics: enable the world, give sprites bodies, set velocity/acceleration/gravity, and resolve collisions with colliders, overlaps, groups, and world bounds.

WHEN TO USE

Use when a Phaser game needs movement or collisions — when the user mentions Arcade Physics, this.physics, setVelocity, collider, overlap, gravity, onFloor, or a platformer/top-down controller. For game config, scenes, and the loader use phaser-core.

Invoke: **/phaser-arcade-physics** · or just describe your task

phaser-core

WHAT IT DOES

Set up and debug a Phaser 4 game: the Game config, the Scene lifecycle (init/preload/create/update), the asset loader, cameras, and cross-scene communication.

WHEN TO USE

Use when building or debugging a Phaser game — when the user mentions Phaser, Phaser.Game, Phaser.Scene, preload/create/update, this.load, this.add, or scene transitions. For Arcade Physics movement/collisions use phaser-arcade-physics.

Invoke: **/phaser-core** · or just describe your task

pixijs-rendering

WHAT IT DOES

Build a Pixijs v8 render layer: create the async Application, load textures with Assets, compose the scene graph with Container and Sprite, drive the ticker loop, wire pointer events, and group draws with render groups.

WHEN TO USE

Use when building or debugging Pixijs v8 — when the user mentions Pixijs, Pixi, Application, app.stage, Container, Sprite, Assets.load, app.ticker, or eventMode. Pins the v8 async init() API.

Invoke: **/pixijs-rendering** · or just describe your task

threejs-gltf-loading

WHAT IT DOES

Load glTF/GLB models in three.js with GLTFLoader and play their skinned animations with AnimationMixer, including DRACO/Meshopt-compressed meshes and KTX2 textures.

WHEN TO USE

Use when importing 3D models into three.js — when the user mentions glTF, GLB, GLTFLoader, AnimationMixer, animation clips, DRACOLoader, or "load a 3D model". For scene/camera/renderer setup use threejs-scene-setup; for materials and lights use threejs-materials-lighting.

Invoke: **/threejs-gltf-loading** · or just describe your task

threejs-materials-lighting

WHAT IT DOES

Light and shade a three.js scene: choose materials (MeshStandardMaterial PBR vs unlit MeshBasicMaterial), add ambient/hemisphere/directional/point/spot lights, turn on shadow maps, and use an environment map (IBL) for realistic reflections.

WHEN TO USE

Use when a three.js model looks black, flat, or wrong — when the user mentions three.js materials, MeshStandardMaterial, lights, shadows, envMap, or PBR. For renderer/loop setup use threejs-scene-setup; for loading models use threejs-gltf-loading.

Invoke: **/threejs-materials-lighting** · or just describe your task

threejs-scene-setup

WHAT IT DOES

Stand up a three.js scene: import maps and the three/addons path, the Scene/PerspectiveCamera/WebGLRenderer trio, the setAnimationLoop render loop, responsive resize, and OrbitControls.

WHEN TO USE

Use when starting or debugging a three.js app — when the user mentions three.js, THREE.Scene, WebGLRenderer, PerspectiveCamera, the render loop, resizing, or OrbitControls. For models use threejs-gltf-loading; for materials/lights use threejs-materials-lighting.

Invoke: **/threejs-scene-setup** · or just describe your task

Other engines

Bevy, pygame, LÖVE and Roblox. · 10 skills

bevy-ecs

WHAT IT DOES

Structure a Bevy app around its Entity Component System: build the App with plugins, define Component/Resource types, write systems with Query/Res/Commands, filter and order systems, and use the Time resource for frame-rate-independent motion.

WHEN TO USE

Use when building or debugging a Bevy game in Rust — when the user mentions Bevy, ECS, App::new, add_systems, Query, Commands, components/systems, or a Cargo.toml depending on bevy.

Invoke: **/bevy-ecs** · or just describe your task

love2d-core

WHAT IT DOES

Structure and debug a LÖVE (Love2D) game in Lua: the love.load/update/draw loop, delta-time movement, input, and screen states.

WHEN TO USE

Use when building a LÖVE 11.x game (main.lua, conf.lua, .love).

Invoke: **/love2d-core** · or just describe your task

pygame-core

WHAT IT DOES

Structure a pygame (pygame-ce) game in Python: the init/event/update/draw loop, delta-time movement, Surface/Rect blitting, keyboard/mouse input, and Sprite/Group management with collision.

WHEN TO USE

Use when building or debugging a pygame game — when the user mentions pygame, pygame-ce, the game loop, blit, Surface, Rect, sprite groups, or clock.tick. Targets pygame-ce.

Invoke: **/pygame-core** · or just describe your task

roblox-characters

WHAT IT DOES

Build respawn-safe Roblox character systems around Players, CharacterAdded/CharacterRemoving, Humanoid, HumanoidRootPart, Animator, R6/R15 rigs, movement, animations and markers, death, tools, accessories, ownership, and custom characters.

WHEN TO USE

Use when character scripts break after respawn, cache stale Humanoids, control movement or velocity, load AnimationTracks, handle death, equip Tools, modify avatars, or support custom player rigs.

Invoke: **/roblox-characters** · or just describe your task

roblox-datastores

WHAT IT DOES

Persist player data in Roblox with DataStoreService: GetDataStore, GetAsync/SetAsync/UpdateAsync/IncrementAsync wrapped in pcall, load-on-join and save-on-leave plus BindToClose, retries, and OrderedDataStore leaderboards.

WHEN TO USE

Use when saving or loading persistent data in a Roblox experience — when the user mentions DataStore, DataStoreService, GetAsync, SetAsync, UpdateAsync, save player data, or leaderboards. For general Luau scripting use roblox-luau.

Invoke: **/roblox-datastores** · or just describe your task

roblox-luau

WHAT IT DOES

Script a Roblox experience in Luau: get services, create and parent Instances, connect events, run server Scripts vs client LocalScripts, and communicate across the client/server boundary with RemoteEvents/RemoteFunctions (server-authoritative).

WHEN TO USE

Use when building or debugging Roblox Studio scripts — when the user mentions Roblox, Luau, services, RemoteEvent, Instance.new, PlayerAdded, or client vs server. For saving player data use roblox-datastores.

Invoke: **/roblox-luau** · or just describe your task

roblox-networking

WHAT IT DOES

Design and harden Roblox client/server networking with RemoteEvent, RemoteFunction, and UnreliableRemoteEvent; server authority, argument and Instance validation, rate limits, proximity/ownership checks, targeted replication, streaming, lifecycle, prediction, and reconciliation.

WHEN TO USE

Use for Roblox remotes, exploits, request spam, multiplayer replication, network ownership, high-frequency cosmetic updates, or server/client desynchronization.

Invoke: **/roblox-networking** · or just describe your task

roblox-physics

WHAT IT DOES

Implement Roblox physical simulation and queries with assemblies, anchoring, constraints, collision groups, CanCollide/CanTouch/CanQuery, raycasts and overlap queries, mass, impulses, forces, velocity, moving assemblies, cleanup, and network ownership.

WHEN TO USE

Use for Roblox collisions, hit detection, RaycastParams, PhysicsService, projectiles, vehicles, knockback, constraints, deprecated BodyMovers, unstable motion, or client-owned physics exploits.

Invoke: **/roblox-physics** · or just describe your task

roblox-studio-workflow

WHAT IT DOES

Modify living Roblox Studio projects safely: inspect Explorer and execution locations, preserve existing structure, reuse modules/remotes, create meaningful Instances, apply Attributes and CollectionService tags, test server/client and multiple devices, inspect Output, and remove experiments/debug objects.

WHEN TO USE

Use when an agent edits a place or rbxl/rbxlx/Rojo project, operates in Studio, adds Scripts/LocalScripts/ModuleScripts, or must verify a Roblox change in context.

Invoke: **/roblox-studio-workflow** · or just describe your task

roblox-ui

WHAT IT DOES

Build production Roblox interfaces with ScreenGui/PlayerGui lifecycle, responsive UDim2 layouts, safe insets, reusable editable Instances, cross-device input and selection, restrained motion, cleanup, and multi-viewport verification.

WHEN TO USE

Use for Roblox HUDs, inventories, shops, settings, ability bars, ScrollingFrames, AutomaticSize, GuiService, gamepad focus, touch UI, or UI that clips, resets on respawn, looks generic, or was generated as one giant LocalScript.

Invoke: **/roblox-ui** · or just describe your task

Disciplines (engine-neutral)

Cross-engine game-dev craft: feel, cameras, AI, audio, shaders, saving and more. · 15 skills

ai-behavior-trees-utility-ai

WHAT IT DOES

Build a production behavior-tree runtime (Blackboard, action/condition leaves, sequence/selector/parallel composites, decorators) and a Utility AI system (response curves — linear, exponential, sigmoid, quadratic — considerations, and action evaluators), plus hybrid BT-drives-Utility agents.

WHEN TO USE

Use when implementing a reusable behavior-tree or utility-based decision system, or tuning enemy/NPC decisions beyond a simple FSM, or when the user mentions behavior tree, blackboard, decorator, selector, sequence, tick status, utility AI, response/scoring curve, or consideration. For choosing between FSM/BT/steering or for pathfinding, use game-ai; for Unreal's BehaviorTree/Blackboard assets, use unreal-behavior-trees.

Invoke: **/ai-behavior-trees-utility-ai** · or just describe your task

audio-design

WHAT IT DOES

Implement game audio practice — bus/mixer architecture and gain in decibels, ducking (sidechain), adaptive/dynamic music via layering and re-sequencing, SFX variation, and beat synchronization. Engine-neutral.

WHEN TO USE

Use when the user mentions audio mixing, audio buses, adaptive/dynamic music, ducking, SFX variation, music layers, or syncing gameplay to the beat.

Invoke: **/audio-design** · or just describe your task

camera-systems

WHAT IT DOES

Build game cameras that feel good — 2D follow with a deadzone, look-ahead, smoothing, and level-bounds clamping; 3D third-person orbit with collision and first-person look; plus multi-target framing and a shake hook. Engine-neutral techniques that pair with the engine's camera node and rigs like Unity Cinemachine or Godot Camera2D/PhantomCamera.

WHEN TO USE

Use when the user mentions camera follow, follow camera, deadzone, look-ahead, camera smoothing, camera bounds/ limits, third-person camera, orbit camera, first-person look, Cinemachine, or camera jitter.

Invoke: **/camera-systems** · or just describe your task

create-game-assets

WHAT IT DOES

Plan, generate, source, normalize, and validate cohesive visual game assets.

WHEN TO USE

Use for art direction, style bibles, sprites, tilesets, backgrounds, UI art, icons, textures, concept art, or 3D asset briefs.

Invoke: **/create-game-assets** · or just describe your task

dialogue-systems

WHAT IT DOES

Build branching dialogue and narrative — a node/choice graph with conditions, variables, and localization hooks — and choose between authoring tools Ink and Yarn Spinner or a custom data-driven runner. Engine-neutral.

WHEN TO USE

Use when the user mentions dialogue system, branching dialogue, conversation tree, choices, Ink (.ink), Yarn Spinner (.yarn), or NPC dialogue.

Invoke: **/dialogue-systems** · or just describe your task

game-ai

WHAT IT DOES

Design NPC and enemy decision-making with finite state machines, behavior trees, steering behaviors, and A* pathfinding — engine-neutral algorithms that pair with the detected engine's navigation API.

WHEN TO USE

Use when building enemy AI, an FSM or behavior tree, steering/flocking, or pathfinding, or when the user mentions state machine, behavior tree, blackboard, A*, navmesh, seek, or patrol/chase.

Invoke: **/game-ai** · or just describe your task

game-feel

WHAT IT DOES

Add "juice" and game feel that makes actions satisfying — screen shake, hit-stop/freeze frames, tweened/eased motion, squash & stretch, knockback, and layered audio-visual feedback — as engine-neutral techniques that pair with the detected engine's tween, particle, and camera APIs.

WHEN TO USE

Use when the user mentions game feel, juice, "make it feel good/punchy", screen shake, hit stop, screen freeze, easing, squash and stretch, impact frames, or feedback/polish on hits, jumps, pickups, and deaths.

Invoke: **/game-feel** · or just describe your task

game-ui-ux

WHAT IT DOES

Design and build game UI/UX — HUDs, menus, and overlays — that survive every screen: anchor-based responsive layout, resolution/aspect scaling and safe areas, keyboard/gamepad focus navigation, a screen/menu state stack, and event-driven (not polled) HUD updates. Engine-neutral patterns that pair with the detected engine's UI skill.

WHEN TO USE

Use when the user mentions HUD, health bar, main menu, pause menu, settings screen, UI layout, anchors, UI scaling, aspect ratio, safe area, controller/keyboard menu navigation, or wiring UI to game state.

Invoke: **/game-ui-ux** · or just describe your task

input-systems

WHAT IT DOES

Architect game input — action mapping (abstracting keys into named actions), rebinding with conflict detection and persistence, multi-device support (keyboard, gamepad, touch), analog deadzones, and feel features like input buffering and coyote time, plus accessibility. Engine-neutral.

WHEN TO USE

Use when the user mentions input mapping, rebind controls, gamepad support, deadzone, input buffering, coyote time, or accessible controls.

Invoke: **/input-systems** · or just describe your task

level-design

WHAT IT DOES

Design and build playable levels — the blockout/whitebox-to-playable workflow, player metrics and grid layout, pacing and flow (tension/rest curve), gating and the critical path, and encounter design. Engine-neutral practice.

WHEN TO USE

Use when the user mentions level design, blockout/whitebox/greybox, level layout, level pacing, encounter design, or the critical path through a level.

Invoke: **/level-design** · or just describe your task

performance-optimization

WHAT IT DOES

Find and fix game performance problems methodically — measure with the engine profiler first, reason about the frame-time budget, locate the CPU-vs-GPU bottleneck, then apply the right fix: object pooling, draw-call batching, fewer allocations/GC spikes, and asset budgets. Engine- neutral method that pairs with each engine's profiler.

WHEN TO USE

Use when the user mentions performance, optimize, low/dropping FPS, frame drops, stutter, lag, profiler, frame budget, draw calls, batching, garbage collection/GC spikes, object pooling, or "the game runs slow".

Invoke: **/performance-optimization** · or just describe your task

physics-tuning

WHAT IT DOES

Tune game physics for stable, good-feeling motion — fixed vs variable timestep, render interpolation, mass/gravity/drag, continuous collision detection (CCD) to stop tunneling, fixing jitter, and collision layers/masks. Engine-neutral.

WHEN TO USE

Use when the user mentions physics feel, jitter, tunneling, fixed timestep, FixedUpdate, CCD, bouncing/unstable physics, or collision layers.

Invoke: **/physics-tuning** · or just describe your task

procedural-gen

WHAT IT DOES

Generate game content procedurally — seeded deterministic RNG, value/Perlin/ Simplex noise for terrain and heightmaps, grid dungeon generation (rooms + corridors, BSP, random walk), and weighted loot/drop tables. Engine-neutral algorithms.

WHEN TO USE

Use when the user mentions procedural generation, perlin/simplex noise, random seed, dungeon generator, heightmap/terrain, or loot tables.

Invoke: **/procedural-gen** · or just describe your task

save-systems

WHAT IT DOES

Design save/load for game state — choosing what to serialize, file formats, save slots, atomic crash-safe writes, schema versioning and migration, and autosave. Engine-neutral.

WHEN TO USE

Use when the user mentions save system, save/load, game state persistence, save slots, autosave, save file corruption, or migrating old saves to a new version.

Invoke: **/save-systems** · or just describe your task

shader-programming

WHAT IT DOES

Write game shaders from cross-engine fundamentals — the vertex→fragment pipeline, coordinate spaces, UV math, and common 2D/3D effects (tint, UV scroll, dissolve, outline, fresnel rim, vignette) in GLSL with HLSL equivalents.

WHEN TO USE

Use when the user mentions shaders, fragment/pixel shader, vertex shader, UV, GLSL, HLSL, or effects like dissolve, outline, or rim light.

Invoke: **/shader-programming** · or just describe your task

Genres

Blueprints for common game types. · 9 skills

card-game

WHAT IT DOES

Build a card game: card data, deck/hand/discard zones, draw/shuffle/reshuffle, a turn structure, costs, and effect resolution.

WHEN TO USE

Use for a deckbuilder, TCG/CCG, or roguelike deckbuilder.

Invoke: **/card-game** · or just describe your task

fps-shooter

WHAT IT DOES

Build a first-person shooter: move+mouse-look controller, hitscan or projectile shooting, weapons, health, and enemy AI.

WHEN TO USE

Use for an FPS, or tuning aim feel, time-to-kill, recoil, or spread.

Invoke: **/fps-shooter** · or just describe your task

platformer

WHAT IT DOES

Build a 2D platformer: run/jump control with coyote time, jump buffering, and variable jump height, plus tiled levels and hazards.

WHEN TO USE

Use for a platformer or Mario/Celeste-like, or tuning jump feel.

Invoke: **/platformer** · or just describe your task

puzzle

WHAT IT DOES

Build a puzzle game: grid/board state, move input, rule-based resolution (match-3 cascades, sokoban pushes, tile logic), scoring, and undo.

WHEN TO USE

Use for a match-3, sokoban, or grid-logic puzzle.

Invoke: **/puzzle** · or just describe your task

roguelike

WHAT IT DOES

Build a roguelike: turn-based grid movement, procedural dungeons, permadeath, field-of-view, and loot tables.

WHEN TO USE

Use for a roguelike/roguelite or turn-based grid dungeon crawler with procedural levels.

Invoke: **/roguelike** · or just describe your task

rpg

WHAT IT DOES

Build an RPG: stats and leveling, inventory and equipment, quests, branching dialogue, save/load, and combat.

WHEN TO USE

Use for an RPG/JRPG, or designing stat, inventory, quest, or combat systems.

Invoke: **/rpg** · or just describe your task

survival-crafting

WHAT IT DOES

Build a survival-crafting game: resource gathering, inventory, crafting and a tech tree, needs (hunger/thirst/temperature), and base building.

WHEN TO USE

Use for a survival or crafting/base-building game.

Invoke: **/survival-crafting** · or just describe your task

tower-defense

WHAT IT DOES

Build a tower defense: enemies pathing along lanes, wave spawning, towers that auto-target and fire, an economy, and lives.

WHEN TO USE

Use for a tower-defense/wave-defense game, or balancing waves and economy.

Invoke: **/tower-defense** · or just describe your task

visual-novel

WHAT IT DOES

Build a visual novel: a branching script, character and background display, a text box with choices, save/load, backlog, and skip/auto.

WHEN TO USE

Use for a VN, dating sim, or branching story game.

Invoke: **/visual-novel** · or just describe your task

Workflows

Prototyping, jams and shipping to stores. · 4 skills

game-jam

WHAT IT DOES

Plan and ship a game under a jam deadline: lock scope to the clock, schedule the hours, cut features, and submit on time.

WHEN TO USE

Use for a game jam (Ludum Dare, GMTK Jam, Global Game Jam), a 48-hour or weekend build, or scoping and submitting a jam entry.

Invoke: **/game-jam** · or just describe your task

itch-publish

WHAT IT DOES

Publish and update a game on itch.io: create the project page and upload builds with the butler CLI (butler push) to named channels.

WHEN TO USE

Use for itch.io publishing, butler push, channel naming for Windows/macOS/Linux/HTML5, versioning uploads, or shipping a jam or release build to itch.io.

Invoke: **/itch-publish** · or just describe your task

prototype-fast

WHAT IT DOES

Build a playable prototype in about an hour to answer one question — is it fun? — with greybox primitives, a hard timebox, and explicit keep/kill criteria.

WHEN TO USE

Use when prototyping a mechanic, making a vertical slice or MVP, greyboxing/blockout, or judging throwaway vs keep.

Invoke: **/prototype-fast** · or just describe your task

steam-publish

WHAT IT DOES

Publish or update a game on Steam with Steamworks and SteamPipe: configure depots and packages, upload builds with steamcmd, set a build live on a branch, and run the release checklists.

WHEN TO USE

Use for Steam publishing, app_build.vdf/steamcmd uploads, depots, beta branches, or a store page release.

Invoke: **/steam-publish** · or just describe your task